implementation as a black box but want to customise the communication only as per their needs. The BIO interface can be found at *$(ROOT)/include/openssl/bio.h*

There are two types of BIOs—a source-sink BIO and a filter BIO. The basic communication mechanisms—for example, communication over a socket or file (yes, TLS records can be written to and read from a file descriptor; remember, this could be a device file too)—can be overridden with a source-sink BIO. As the name suggests, this acts as the source of data on the sending end and the sink on the receiving end. For filtering, buffering and translation activities, you can use a filter BIO. As you can see, you can pick and choose one source sink BIO, many filter BIOs and stack up BIOs, one on top of the other, like building blocks.

A BIO is overridden by registering a structure called BIO_METHOD, which contains hooks for different I/O operations. Please find the snippet from *$(ROOT)/include/openssl/bio.h* to see the BIO methods provided by OpenSSL:

```
BIO_METHOD *BIO_s_mem(void);
BIO *BIO_new_mem_buf(void *buf, int len);
BIO_METHOD *BIO_s_socket(void);
BIO_METHOD *BIO_s_connect(void);
BIO_METHOD *BIO_s_accept(void);
BIO_METHOD *BIO_s_fd(void);
#ifndef OPENSSL_SYS_OS2
BIO_METHOD *BIO_s_log(void);
#endif
BIO_METHOD *BIO_s_bio(void);
BIO_METHOD *BIO_s_null(void);
BIO_METHOD *BIO_f_null(void);
BIO_METHOD *BIO_f_buffer(void);
#ifdef OPENSSL_SYS_VMS
BIO_METHOD *BIO_f_linebuffer(void);
#endif
BIO_METHOD *BIO_f_nbio_test(void);
#ifndef OPENSSL_NO_DGRAM
BIO_METHOD *BIO_s_datagram(void);
#ifndef OPENSSL_NO_SCTP
BIO_METHOD *BIO_s_datagram_sctp(void);
```

You may have noticed that the naming convention for the source sink BIO is BIO_s_* and for a filter BIO, it is BIO_f_*. These APIs return BIO_METHOD structures containing hooks, which implement the behaviour for each of these BIOs. These BIO methods are implemented by OpenSSL itself and are used for its default communication as well. So, the core functionality of a BIO is in its BIO_METHOD.

Now, let's look at how OpenSSL implements these BIOs. We will look at how a basic socket connection is implemented in OpenSSL and then at how we can override the default.

If you take a look at *$(ROOT)/crypto/bio/bss_conn.c*, you can see the BIO_METHOD structure that contains hooks for different operations on a socket connection. The code snippet is as follows:

```
static BIO_METHOD methods_connectp=
{
        BIO_TYPE_CONNECT,
        "socket connect",
        conn_write,
        conn_read,
        conn_puts,
        NULL, /* connect_gets, */
        conn_ctrl,
        conn_new,
        conn_free,
        conn_callback_ctrl,
};
```

In this code, BIO_TYPE_CONNECT specifies that it is a connection BIO. The function *conn_write* implements the functionality of writing to a socket, while *conn_read* implements reading from a socket. *conn_ctrl* implements control plane operations, like the current state of the machines in the whole TLS process (the implementations of these functions are also present in the same file).

As explained earlier, this is the default implementation of OpenSSL. To override this, we need to implement our own hooks. Given below is a code snippet that implements our user defined BIO method and registers it with OpenSSL's BIO interface:

```
static BIO_METHOD methods_connect= {
    BIO_TYPE_CONNECT,
    "Connect overrides",
    l_bio_WriteCallback, /* This is a locally implemented
callback for write operations */
    l_bio_ReadCallback, /* This is a locally implemented
callback for read operations */
    l_bio_PutsCallback, /* Likewise for puts */
    NULL,
    l_bio_CtrlCallback, /* This is called to override/log
commands being called */
    l_bio_NewCallback,
    l_bio_FreeCallback
};

/* Initialize OpenSSL library */
static void
l_InitOpenSSL()
{
    ERR_load_crypto_strings();
    ERR_load_SSL_strings();
    OpenSSL_add_all_algorithms();
    SSL_library_init();
    SSL_load_error_strings();
}
```